

Towards contextual, cost-efficient predictive maintenance in heavy-duty trucks

Louis Carpentier*, Arne De Temmerman*, and Mathias Verbeke

Department of Computer Science, KU Leuven, Belgium
Flanders Make@KU Leuven, Belgium

{louis.carpentier, arne.detemmerman, mathias.verbeke}@kuleuven.be

Abstract. Predictive maintenance is a crucial yet challenging task in many industrial applications. This work explores a large repository of existing techniques and approaches to process historical data and predict if an asset is at risk of failure. In particular, the operational condition and specification of Scania trucks in heavy-duty applications is considered as part of the IDA 2024 Industrial Challenge.

Keywords: Predictive maintenance · Time series analysis · Contextual decision-making · Feature extraction · Survival analysis

1 Introduction

Predictive maintenance, i.e., predicting whether an industrial asset is at risk of imminent failure, is a crucial yet challenging task in many industrial applications. Prediction of future failure allows maintenance to be planned before the fault happens, which in turn reduces the downtime [4]. Monitoring capabilities tremendously increased due to the paradigm shift towards Industry 4.0, enabling the development of automated decision-making processes.

Specifically, we look at predicting the failure of an anonymized component in Scania trucks in heavy-duty applications as part of the IDA 2024 Industrial Challenge [3]. The dataset includes variables capturing the operational condition of the trucks over time and the truck specification. In this work, we explore a wide range of techniques and methods to predict upcoming component failures for condition monitoring in general and for Scania trucks in particular. The main contributions of this work are as follows:

1. Transformation of observational readouts into history-aware feature vectors;
2. Application of inherently different models for predictive maintenance based on classification, regression, and survival analysis;
3. Open-source implementation of the methodology, including the source code, notebooks, and raw experimental results, to ensure reproducibility¹.

* Both authors contributed equally to the paper

¹ <https://gitlab.kuleuven.be/u0158714/ida-industrial-challenge-2024>

2 Problem formulation

The train, validation, and test datasets consist of 33 651 vehicles in total with the following information:

1. **Operational readouts.** Onboard sensors monitor crucial parameters regarding the vehicle’s real-time condition and performance. To reduce storage requirements, the data is stored locally in histograms. In particular, multiple ranges or bins are defined for each variable, and each bin has one counter. Whenever a value is measured in a certain bin, the corresponding counter is incremented. The operational readouts are snapshots of these counters taken at irregular time intervals. Note that the operational readouts for each variable are monotonically increasing because the counters are only incremented. In total, 14 parameters were selected with varying numbers of bins, resulting in 105 attributes. Figure 1 plots the ten bins of feature 158 in function of the timestep for vehicle 100. Table 1 lists the total number of vehicles and readouts together with the average number of readouts per vehicle and their missing values. For each vehicle in the validation and test set, a random observation is selected, and only the earlier readouts are provided to simulate the usage of a prediction model in a real-world setting.

	Train set	Validation set	Test set
#vehicles	23 550	5046	5045
%vehicles	70%	15%	15%
#readouts	1 122 452	196 227	198 140
#readouts per vehicle (average)	48	46	45
#missing values	354 634	60 339	66 403
%missing values	0.0030%	0.0029%	0.0032%

Table 1: Size and amount of missing values in the train, validation, and test sets

2. **Specification.** The specification of a truck corresponds to its model, including information such as engine type, weight capacities, and other technical details. There are eight categorical features with a variable number of categories, which totals in 11 181 240 possible specifications, of which only 3 607 occur in the training data.

The train set also contains a time-to-event table, indicating if and when a vehicle had a defect. The last observation of each vehicle in the validation set is assigned 1 of 5 classes, indicating when the vehicle will fail: class 0 instances will not fail in the next 48 time steps, class 1 instances will fail within 48 to 24 time steps, class 2 instances within 24 to 12 time steps, class 3 instances within 12 to 6 time steps, and class 4 instances will fail within 6 time steps. Thus, classes 1, 2, 3 or 4 indicate failure with increasing severity. The goal of the IDA challenge is to predict these labels for the last observation of each vehicle in the test set.

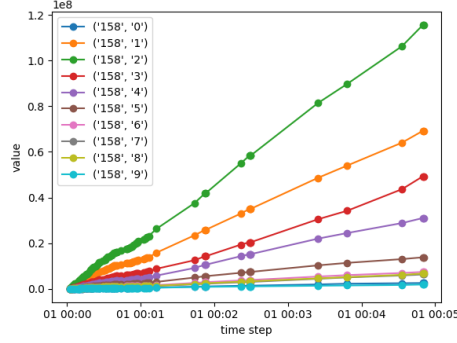


Fig. 1: Sample of the ten bins for feature 158 in operational readout data in function of the time step for vehicle 100

The costs associated with predictive maintenance are highly unbalanced, caused by a larger cost to repatriate and repair a defective vehicle than to replace a non-defective component pre-emptively. Therefore, a cost function is defined by domain experts, which assigns a cost $C_{i,j}$ for predicting j if the actual label is i . These costs are shown in Table 2. In general, $C_{i,j}$ indicates the cost of a false positive if $i < j$, and the cost of a false negative if $i > j$. Consequently, false negatives have a much higher cost. For some model M , denote $N_{i,j}$ as the number of times that M predicted class i if the actual label was j . Then, the summarized cost of M is computed according to Equation 1.

$$\text{cost}(M) = \sum_{i=0}^4 \sum_{j=0}^4 C_{i,j} \times N_{i,j} \quad (1)$$

	Predicted: 0	Predicted: 1	Predicted: 2	Predicted: 3	Predicted: 4
Actual: 0	$C_{0,0} = 0$	$C_{0,1} = 7$	$C_{0,2} = 8$	$C_{0,3} = 9$	$C_{0,4} = 10$
Actual: 1	$C_{1,0} = 200$	$C_{1,1} = 0$	$C_{1,2} = 7$	$C_{1,3} = 8$	$C_{1,4} = 9$
Actual: 2	$C_{2,0} = 300$	$C_{2,1} = 200$	$C_{2,2} = 0$	$C_{2,3} = 7$	$C_{2,4} = 8$
Actual: 3	$C_{3,0} = 400$	$C_{3,1} = 300$	$C_{3,2} = 200$	$C_{3,3} = 0$	$C_{3,4} = 7$
Actual: 4	$C_{4,0} = 500$	$C_{4,1} = 400$	$C_{4,2} = 300$	$C_{4,3} = 400$	$C_{4,4} = 0$

Table 2: Table of prediction costs

3 Methodology

3.1 Data transformation

The non-uniform time-series from the operational readouts of the train, validation, and test dataset were processed to a) handle the missing values, b) produce rolling windows based on the time step, c) extract time-series features from these

created indexes of the windows to the labels. The *fixed-time index-based window* creates a window before each readout with timespan w , resulting in a less sparse dataset with a one-on-one corresponding label.

For each type of window, a wide range of features was extracted (see Appendix A) through `tsfresh` [2], resulting in 652 features for each window size and attribute, totaling to 342 300 calculated features.

This procedure is applied to each readout of the train set, whereas only the features for the last readout need to be calculated for the validation and test set.

Feature selection We removed irrelevant features using Kendall’s τ_B . For some feature f , a pair of values f_i and f_j with corresponding labels y_i and y_j is said to tie if either the feature values or the labels are equal. If both $f_i > f_j$ and $y_i > y_j$ or both $f_i < f_j$ and $y_i < y_j$, then the pair is said to be concordant, otherwise, it is discordant. For each f , Denote with P the number of concordant pairs, Q the number of discordant pairs, T the number pairs with $f_i = f_j$ and U the number of pairs with $y_i = y_j$. Then, τ_B of feature f is computed according to Equation 2.

The null hypothesis for Kendall’s Tau states that there is no association between the two variables. The p-value represents the probability of observing a value of Tau as extreme as, or more extreme than, the one calculated from the sample data, under the assumption that the null hypothesis is true (Equation 3). We varied the threshold on the resulting p-values between 0.0005, 0.001, 0.005, 0.01, and 0.05 to create multiple datasets with a variable number of relevant features. At the end, a threshold of 0.01 was used.

$$\tau_B = \frac{(P - Q)}{\sqrt{(P + Q + T) \times (P + Q + U)}} \quad (2)$$

$$p = P(|\tau_B| \geq |\tau_{\text{observed}}|) \quad (3)$$

Besides univariate feature selection, the Pearson correlation coefficient $\rho_{i,j}$ was used to remove highly collinear features f_i and f_j , and is computed as $\rho_{i,j} = \text{cov}(f_i, f_j) / (\sigma_{f_i} \cdot \sigma_{f_j})$. For two correlated features, we removed the feature with the smallest τ_k . Here, the threshold on $\rho_{i,j}$ was set to 0.5.

Label transformation Train labels can be inferred from the time-to-event table. If a vehicle was repaired at time t , all its readouts with a time step in $[t - 6, t]$ are of class 4, those in $[t - 12, t - 6]$ in class 3, and so on. Additionally, for a healthy vehicle that was studied until time t , we removed the readouts in the interval $[t - 48, t]$ because it might fail immediately after the monitored period.

Resampling for improved class representation The dataset is highly imbalanced because failures are rare. Specifically, there are 1 096 712 instances of class 0, 12 503 of class 1, 6 179 of class 2, 3 200 of class 3, and 3 858 of class 4.

Training on this data introduces extreme bias in the models. To cope with this, we undersample class 0 such that there are an equal number of healthy instances as instances of class 1, the largest non-healthy class.

3.2 Predictive model

Due to the openness of the training labels, multiple inherently different models were used to predict component failure. In particular, we used the implementations from `scikit-learn` [5] and `XGBoost` [1]. More details regarding the hyperparameters can be found in Appendix C.

Classification models The data can be treated as a multiclass classification problem, with one healthy state and four failure states. We trained K-NN, Decision Tree, Random Forest, Logistic Regression, Support Vector Machine, and XGBoost classifiers to predict the class labels directly.

Regression models Classification models typically treat each class independently, while the classes in this dataset represent varying levels of severity. This knowledge is encoded by treating the discrete class labels as regression values. Then, we trained several regression models, including K-NN, Decision Tree, Random Forest, Linear, Support Vector Machine, and XGBoost regressors. The models' continuous predictions were rounded to derive discrete class labels. We also experimented with using a ceiling function to convert predictions into class labels, addressing the imbalanced cost function in Equation 1, but this did not significantly impact the results.

Survival analysis The train data contains the failure time of a component, providing more fine-grained information than the five discrete labels. Therefore, we trained a survival analysis model that predicts the time until the component fails, which can subsequently be translated to class labels. In particular, we used XGBoosting with the Accelerated Failure Time model.

Including truck specification Besides historical observations, the data also includes vehicle specifications, which the models incorporate as contextual information. Specifically, the distance between two vehicles is computed as a convex combination between the number of matches in the vehicle specification, on the one hand, and the Euclidean Distance of the min-max normalized trend of the operational readouts, on the other hand. Hierarchical clustering with average linkage is used to cluster the vehicles, which enables to train a model within each cluster or context. During prediction, we check which cluster the vehicle specification belongs to and use the corresponding model. Additionally, a global, non-contextual model is trained using all data, which predicts the label for vehicles that do not belong to any context.

4 Evaluation

The validation set enables a quantitative comparison of the different models but does not indicate if the models generalize properly to unseen data. Therefore, we included several baselines:

1. **Always predict X.** A model that always predicts a single label, independent of the observational readouts or vehicle specification;
2. **Random.** Predict a random label for each vehicle. This is done 100 times, of which the average cost is taken to reduce variance;
3. **Weighted random.** Similar to random sampling, but a weight is assigned to each class label based on the number of occurrences of each class in the train data, thus taking class imbalance into account.

The prediction cost on the validation set, according to Equation 1, over the number of vehicles in the validation set is shown in Figure 3. Multiple conclusions can be drawn from this figure.

- Baseline *Always predict 4* gives a comparatively low average cost per vehicle. This is due to the highly imbalanced cost function. A model that overestimates the actual time until failure is heavily penalized, while a model that underestimates this time is penalized less. Although the healthy instances significantly outnumber the unhealthy instances, always predicting 4 results in a relative small cost on the validation set.
- Besides this baseline, the survival analysis models give the best results. Depending on the chosen sampling method for the train set, the model initially produces representative predictions. However, the predictions do not supersede *Always predict 4* and converge to this prediction towards the later boosting rounds. Including contextual information does not affect this.
- Classification models treat the different classes independently, while the regression models encode the severity levels. Nevertheless, in general, the classification models have a higher performance than the regression models.

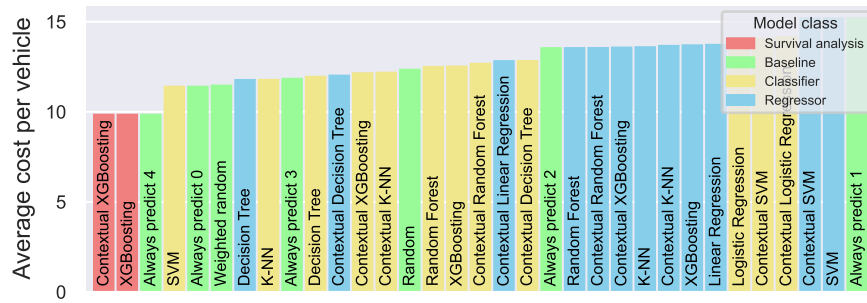


Fig. 3: Average prediction cost for the different models on the validation set

- Generally speaking, including the specification as context does not increase the performance. This is especially true for the SVM classifier, for which the average cost increases from 11.38 to 14.02 by the inclusion of context. However, for linear regression, the cost decreases by including context, from 13.70 to 12.80. This is because the full dataset is not linear, but data within a context follows a more linear trend.

5 Conclusion

This work explored a wide range of different models to predict if a component is at risk of imminent failure, specifically for Scania trucks in heavy-duty applications [3]. We showed that this task is non-trivial due to the highly imbalanced dataset and complex feature space. Therefore, we believe out-of-the-box models are insufficient for predictive maintenance, and dedicated and fine-tuned approaches must be developed for each application. Additionally, we illustrate that it is crucial to compare with baselines. By only comparing out-of-the-box models, one only measures the relative performance of different models without actually knowing if the model performs well.

Acknowledgements

This research is supported by Flanders Innovation & Entrepreneurship (VLAIO) through the AI-ICON project CONSCIOUS (HBC.2020.2795), the Flemish government under the Flanders AI Research Program, and by Internal Funds KU Leuven (STG/21/057).

References

1. Chen, T., Guestrin, C.: Xgboost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data mining. pp. 785–794 (2016)
2. Christ, M., Braun, N., Neuffer, J., Kempa-Liehr, A.W.: Time series feature extraction on basis of scalable hypothesis tests (tsfresh—a python package). *Neurocomputing* **307**, 72–77 (2018)
3. Kharazian, Z., Lindgren, T., Magnússon, S., Steinert, O., Reyna, O.A.: SCANIA component x dataset: A real-world multivariate time series dataset for predictive maintenance. *arXiv preprint arXiv:2401.15199* (2024). <https://doi.org/10.48550/arXiv.2401.15199>
4. Li, Z., Wang, K., He, Y.: Industry 4.0 - potentials for predictive maintenance. In: Proceedings of the 6th International Workshop of Advanced Manufacturing and Automation. pp. 42–46. Atlantis Press (2016/11). <https://doi.org/10.2991/iwama-16.2016.8>
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011)

A Extracted features

Feature calculator	Hyperparameter	Range
time reversal asymmetry statistic	lag	[1,4]
c3	lag	[1,4]
cid ce	normalize	{True, False}
symmetry looking	r	[0.05,1.0,0.05]
large standard deviation	r	[0.05,1.0,0.05]
quantile	q	[0.1, 0.9,0.1]
autocorrelation	lag	[1, 10]
agg autocorrelation	maxlag f_agg	40 {mean, median, var}
partial autocorrelation	lag	[1, 10]
number cwt peaks	n	[1, 5]
number peaks	n	{1, 3, 5, 10, 50}
binned entropy	max_bins	[0,10]
index mass quantile	q	[0.1, 0.9,0.]
spkt welch density	coeff	[2, 5, 8]
ar coefficient	coeff k	[0,11] [0,10]
fft coefficient	a k	{real, imag, abs, angle} [0,100]
fft aggregated	s	{centroid, variance, skew, kurtosis}
value count	value	{0, 1, -1}
range count	min,max	{{-1,1},{-1e12,0},{0,1e12}}
linear trend	attr	{pvalue,rvalue,intercept,slope,stderr}
agg linear trend	attr chunk_len f_agg	{rvalue,intercept,slope,stderr} {5,10,50} {max, min, mean, var}
augmented dickey fuller	attr	{teststat,pvalue,usedlag}
number crossing m	m	{0, 1, -1}
energy ratio by chunks	#segments i	10 [0,10]
ratio beyond r sigma	x	{0.5, 1, 1.5, 2, 2.5, 3, 5, 6, 7, 10}
count above	t	0
count below	t	0
lempel ziv complexity	x	{2, 3, 5, 10, 100}
fourier entropy	x	{2, 3, 5, 10, 100}
permutation entropy	τ x	0 {3, 4, 5, 6, 7}
query similarity count	threshold	0.0
matrix profile	threshold f	0.98 {min, max, mean, median, 25, 75}
mean n absolute max	#maxima	{3,5,7}

Table 3: The 33 time-series feature calculators with ranges of used hyperparameters, totaling to 652 calculated features

B Selected features

The 704 selected features can be found in the configuration file.

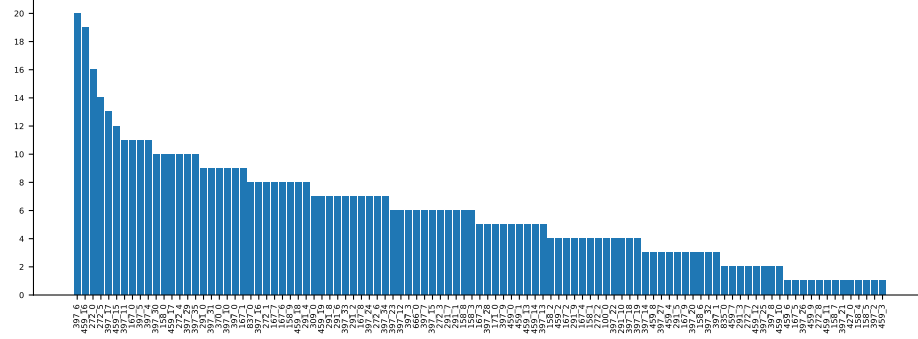


Fig. 4: The count of each column in the selected features

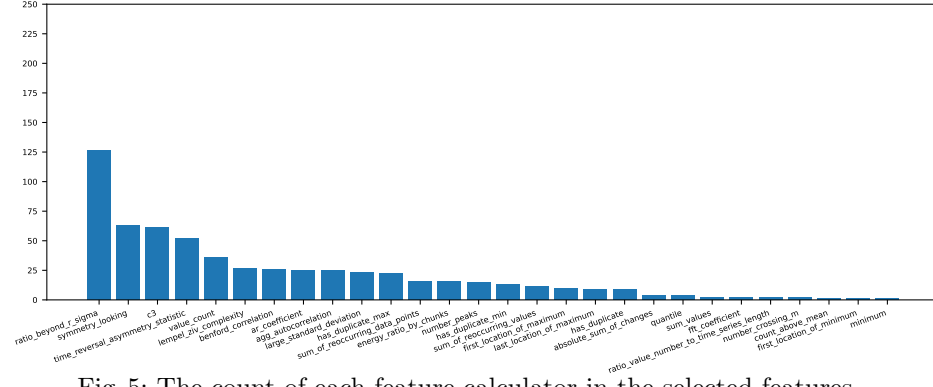


Fig. 5: The count of each feature calculator in the selected features

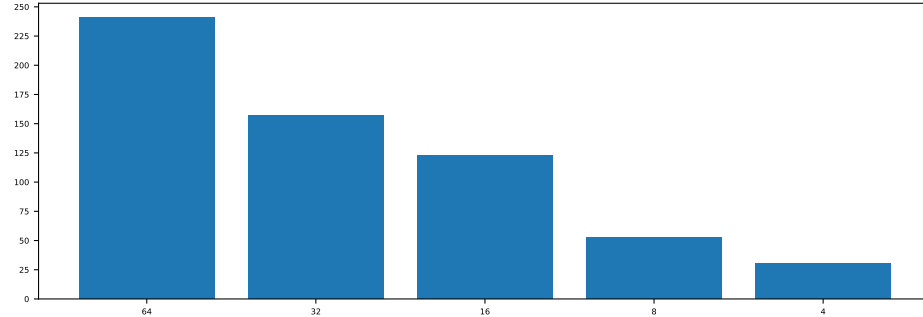


Fig. 6: The count of each feature window in the selected features

C Hyperparameters

The hyperparameters of logistic regression, linear regression, decision tree, and random forest were tuned in this work using grid search. Table 4 lists the used parameter grids. Even though the limited number of values, the tuning process required significant computational resources. Additionally, we noticed that tuning had a small to no effect on the final cost. Therefore, we used the default hyperparameters as provided by the Python libraries. Ultimately, the hyperparameters were fixed as shown in Table 5.

Parameter	Values
penalty	{l1, l2, elasticnet}
C	{0.1, 1, 10}

(a) Logistic Regression

Parameter	Values
fit_intercept	{True, False}

(b) Linear Regression

Parameter	Values
criterion	{squared_error, absolute_error}
max_depth	{None, 5, 9, 12}
min_samples_split	{2, 4, 16}
min_samples_leaf	{1, 4, 16}

(c) Decision Tree

Parameter	Values
n_estimators	{50, 100}
criterion	{squared_error, absolute_error}
max_depth	{5, 9, 12}
min_samples_split	{2, 5}
min_samples_leaf	{1, 2}

(d) Random Forest

Parameter	Values
max_depth	[1,20]
eta	[0, 1]
gamma	[0, 10e1]
reg_alpha	[10e-7, 10]
reg_lambda	[0,1]
colsample_bytree	[0.5,1]
colsample_bynode	[0.5,1]
colsample_bylevel	[0.5,1]
min_child_weight	[1,10]
max_delta_step	[1,10]
subsample	[0.5,1]

(e) XG Boosting for Survival Analysis

Table 4: The parameter grids used for grid search

Parameter	Value
penalty	l2
C	1
fit_intercept	True

(a) Logistic Regression

Parameter	Value
fit_intercept	True

(b) Linear Regression

Parameter	Values
criterion	squared_error
max_depth	None
min_samples_split	2
min_samples_leaf	1

(c) Decision Tree

Parameter	Value
n_estimators	100
criterion	gini
max_depth	None
min_samples_split	2
min_samples_leaf	1

(d) Random Forest

Parameter	Value
kernel	rbf
C	1
gamma	scale

(e) Support Vector Machine

Parameter	Value
n_neighbors	5
weights	uniform
metric	minkowski
p	2

(f) K-NN

Parameter	Value
learning_rate	0.3
min_split_loss	0
max_depth	6
min_child_weight	1
max_delta_step	0
subsample	1
sampling_method	uniform

(g) XG Boosting

Parameter	Value
aft_loss_distribution	normal
aft_loss_distribution_scale	1.2
colsample_bylevel	0.882146
colsample_bynode	0.700088
colsample_bytree	0.523955
eta	0.00572
gamma	43.92145
max_delta_step	3
max_depth	13
min_child_weight	4
objective	survival:aft
reg_alpha	1.929757
reg_lambda	0.899825
subsample	0.654796
tree_method	hist

(h) XG Boosting for Survival Analysis

Table 5: The selected hyper parameters for each model